

NATIONAL ECONOMICS UNIVERSITY
FACULTY OF MATHEMATICAL ECONOMICS

MAJOR ASSIGNMENT

Course: Deep Learning

TOPIC:

**Robust Real-World Waste Classification using Custom
CNN Architectures and Transfer Learning**

Instructor:

Nguyen Thi Kim Ngan

Students:

Nguyen Vinh Khanh

Do Huu Kien

Nguyen Tong Nguyen Vu

Nguyen Phu Nam

Hanoi, October 2026

Part 1: Theory

This part goes through the parts of a convolutional neural network that we actually used in the project. We follow the order of the course slides [1, 2, 3] and, for each idea, point to the place where it shows up in our three models.

1. Convolution, filters and feature maps

In a fully connected network every pixel would need its own weight, which is hopeless for a 224×224 colour image. A convolutional layer gets around this by sliding one small kernel, for example 3×3 , over the whole image and taking a weighted sum at each position. Strictly speaking this is a cross-correlation, which is also how the lecture defines it [1]. The same few weights are reused everywhere, so a kernel that has learned to respond to an edge will find that edge wherever it is.

Each kernel gives one output map, called a feature map. A layer with 64 kernels gives 64 feature maps, and the next layer mixes them into more complex patterns. With an input of size $n_h \times n_w$, a kernel of size $k_h \times k_w$, p_h and p_w rows and columns of padding and strides s_h and s_w , the output height is

$$\left\lfloor \frac{n_h - k_h + p_h + s_h}{s_h} \right\rfloor, \quad (1)$$

and the width follows the same formula [1]. We always use stride 1 and $p_h = k_h - 1$, which keeps the height and width unchanged. In our networks only the pooling layers make the feature maps smaller.

2. Activation functions

A stack of convolutions without anything in between is still a single linear map, so each convolution is followed by a non-linear activation. Our own models use ReLU, $f(x) = \max(0, x)$. It costs almost nothing and its gradient does not vanish for positive inputs. EfficientNet-B0 uses SiLU, $x \cdot \sigma(x)$, which is a smoother version of the same idea. At the very end, the network outputs one score z_k per class, and softmax turns these scores into probabilities:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}. \quad (2)$$

3. Pooling

Pooling layers sit between convolution blocks. Following the lecture, they make the network less sensitive to the exact position of a feature and they shrink the representation [1]. We use 2×2 max pooling with stride 2, which keeps the largest value of every window and halves the height and width. In our simple CNN a 224×224 image is down to 28×28 after three poolings; in the multi-scale CNN it reaches 7×7 after five. After every pooling, each value of the next layer depends on a larger part of the input image (its receptive field), and this turns out to matter for our results.

4. Fully connected layers and global average pooling

The classic recipe flattens the last feature maps into one long vector and passes it to fully connected layers [1]. For our multi-scale CNN that vector would have $7 \times 7 \times 256 = 12,544$ entries per image. We chose global average pooling (GAP) instead, which averages each feature map into one number and so leaves 256 values. GAP has no weights, which also means fewer things to overfit. After it comes a small dense head: Dense(128), ReLU, Dropout and Dense(9) in our own models, and only Dropout and Dense(9) in EfficientNet-B0.

5. Loss and optimisation

All models are trained with the cross-entropy loss over a batch of N images with true classes y_i :

$$L = -\frac{1}{N} \sum_{i=1}^N \log p_{i,y_i}. \quad (3)$$

The gradients come from backpropagation and the update rule is AdamW [9]. AdamW is Adam with weight decay applied directly to the weights instead of being mixed into the gradient. We start from a learning rate of 10^{-3} and halve it whenever the validation macro-F1 has not improved for two epochs. On the GPU we train in mixed precision (16-bit floats), which makes each epoch faster.

6. Regularisation

The lecture on CNN techniques lists overly complex models and small or unrepresentative datasets among the main reasons for poor predictions [3]. Our dataset is small (about 3,300 training images), so we use several regularisers at once. Batch normalisation [7, 2] follows every convolution and keeps activations on a similar

scale, which mainly makes training more stable. Dropout [8] switches off random neurons in the head (30 % in our models, 20 % in EfficientNet-B0). Weight decay of 10^{-4} pulls the weights towards zero. Finally, we stop training once the validation macro-F1 has not improved for 5 epochs and keep the best epoch.

7. Data augmentation

Augmentation creates new training images on the fly by changing the existing ones at random, with the goal that the model never sees exactly the same image twice [3]. We picked mild changes that still give a realistic photo: a crop of 80 to 100 % of the image that is resized back to 224×224 , a horizontal flip, a rotation of at most 10° , and up to 20 % change in brightness, contrast and saturation. We left the hue unchanged, since colour is one of the cues that separate materials such as brown cardboard and grey metal. Validation and test images are only resized and normalised.

8. Transfer learning and fine-tuning

Transfer learning reuses what a network has learned on a large dataset for a new task [3]. The lecture splits this into a pretraining phase on a dataset such as ImageNet, a transfer phase in which the pretrained weights are either kept fixed (feature extraction) or partly retrained, and an optional fine-tuning phase on the small target dataset [3]. Our third model follows this plan closely. We take EfficientNet-B0 pretrained on ImageNet-1K, which has 1,000 classes [10], and first train only a new 9-class head on top of the frozen backbone. After that we unfreeze the last three blocks and continue with a learning rate ten times smaller, so the pretrained features are adjusted gently rather than overwritten.

Part 2: Practical Application

I. Problem Description

- **Title:** Robust Real-World Waste Classification using Custom CNN Architectures and Transfer Learning.
- **Research objectives:** our goal is to see how much a more complex CNN and ImageNet pretraining help on real waste photos when everything else stays the same: the same split, the same preprocessing, the same training rules and the same metrics. We set four research questions:

- RQ1. How well does a simple CNN trained from scratch classify RealWaste images?
- RQ2. Does our own multi-scale CNN do better than the simple CNN under the same data protocol?
- RQ3. How much does ImageNet transfer learning with EfficientNet-B0 add compared with CNNs trained from scratch?
- RQ4. (ablation) Does training-time augmentation help the multi-scale CNN?
- **Input of the problem:** one RGB photo of a single waste item, resized to 224×224 pixels.
 - **Output of the problem:** the predicted class and the probability of each of the 9 classes.
 - **Summary of the tasks performed:** we started with an audit of the downloaded data (class counts, image format, broken files). The first train/validation/test split we made was a random one, but a duplicate check showed that the same object often appears in two or three consecutive photos, so we threw that split away before any training and replaced it with a grouped split. On top of this split we wrote one shared pipeline for loading, augmentation, training and evaluation, and trained the four models in Table 1. Each model was selected on validation macro-F1. Because one run can be lucky or unlucky, the three main models were trained twice more with different random seeds. At the end every selected model was evaluated once on the test set, and we looked at the images it got wrong.

Table 1: Models compared in this project.

ID	Model	Trained from	Answers
E1	SimpleCNN (ours)	scratch	RQ1
E2	MultiScaleCNN (ours)	scratch	RQ2
E3	EfficientNet-B0	ImageNet, fine-tuned	RQ3
E4	MultiScaleCNN, no augmentation	scratch	RQ4

II. Dataset Description

- **Download link:** RealWaste [4] is published in the UCI Machine Learning Repository (<https://archive.ics.uci.edu/dataset/908/realwaste>,

licence CC BY 4.0). We downloaded it from the Kaggle mirror <https://www.kaggle.com/datasets/joebeachcapital/realwaste> (version 1).

- **Description:** the dataset has 4,752 colour photos, each showing one waste item. The items belong to 9 material classes: Cardboard, Food Organics, Glass, Metal, Miscellaneous Trash, Paper, Plastic, Textile Trash and Vegetation. All images are RGB JPEG files of 524×524 pixels, and none of them was broken when we opened them. The classes are not balanced (Figure 1). Plastic has 921 images while Textile Trash has only 318, almost three times fewer, so a model can look accurate overall while doing badly on the small classes. This is why we judge the models mainly by macro-averaged scores.

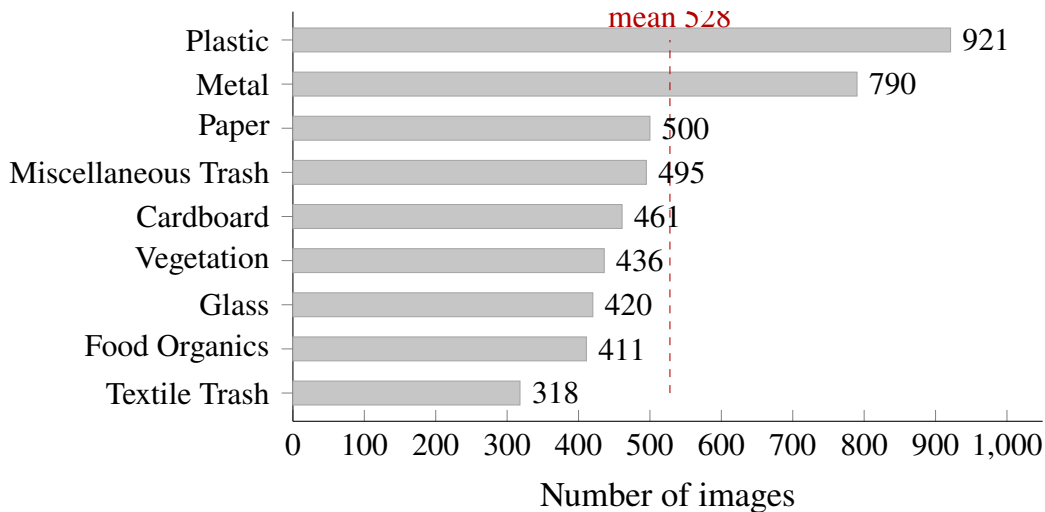


Figure 1: Images per class in our copy of RealWaste (4,752 images). Plastic has 2.9 times as many images as Textile Trash.

When we looked through the images, the first thing we noticed was how uniform they are. Every photo shows one item from above on the same concrete floor, so the background tells the model nothing about the class. What does change is the lighting and colour cast: some photos are greyish, others clearly blue. A few items are hard even for a person. One “Cardboard” photo, for example, is printed food packaging that we would have labelled as Paper or Plastic. Figure 2 shows some examples.



Figure 2: Three sample images per class.

III. CNN Model Design

1. Preprocessing

Cleaning and duplicate check

There was little to clean in the usual sense: all 4,752 files opened without errors and no two files were byte-for-byte identical. The problem we did find was less obvious. To look for near-duplicates we computed a 256-bit difference hash of every

image, a fingerprint that changes little when a photo is re-saved or slightly edited, and inspected the most similar pairs. That turned up only one near-identical pair across subsets. We then also looked at 32 random groups of three images with consecutive file numbers, and in 17 of them the same object had simply been photographed again, usually moved or turned over. The hash had missed these because the photos really are different. Under our first, random split, 4 of those 32 objects had one photo in the training set and another in validation or test, which means the test score would partly reward memorising objects.

Grouped split

Since re-shots almost always have neighbouring file numbers, we split by groups of numbers instead of by single images. Within each class, the files are cut into blocks of ten consecutive numbers (Plastic_640 to Plastic_649 form one block). The blocks are shuffled with seed 42, and whole blocks go to the training, validation or test set in a 70/15/15 ratio per class. Table 2 shows how much this reduced the leakage, and Table 3 gives the final sizes.

Table 2: Leakage checks for the random image-level split we started with and for the grouped split we used.

Check	Random split	Grouped split
Same-class pairs 1 file number apart, in different subsets	46.7 %	4.7 %
Same-class pairs 2 file numbers apart, in different subsets	45.8 %	9.4 %
Smallest hash distance, val/test image to nearest train image	22	29
Spot-check objects with photos in train and in val/test	4 of 32	1 of 32

Table 3: Subsets of the grouped split (seed 42), shared by every model.

Subset	Images	Share	Purpose
Training	3,326	69.99 %	fit the weights
Validation	719	15.13 %	early stopping and checkpoint selection
Test	707	14.88 %	final evaluation, used once
Total	4,752	100 %	

The split is not perfect. In the same 32 groups we still found one object, a heart-shaped ornament photographed front and back as Miscellaneous Trash 59 and 60, that falls on both sides of a block boundary. Re-shots that are far apart in numbering would also slip through. We come back to this in the limitations. To make sure every model saw exactly the same images, the split was saved once and its SHA-256 hash is checked each time it is loaded.

Normalisation

Images are converted to RGB, scaled to $[0, 1]$ and normalised per channel with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). EfficientNet-B0 expects exactly these values. We used them for the two models trained from scratch as well, so that the only difference between the models is the model itself. Because the values are fixed constants, nothing is estimated from our own data, including the test set.

Augmentation (training set only)

Table 4: Training augmentation, applied anew every epoch. Validation and test images are only resized to 224×224 and normalised.

Step	Setting
Random resized crop	80 to 100 % of the image area, output 224×224
Horizontal flip	probability 0.5
Rotation	up to $\pm 10^\circ$
Colour jitter	brightness, contrast and saturation 0.2; hue unchanged

The photos are square, so no cropping is needed at evaluation time. E1, E2 and E3 use the same augmentation. E4 uses none, since it exists only to measure what augmentation does.

2. Simple CNN (E1)

Our first model is meant as a baseline and is kept small on purpose. It has three convolution blocks, each made of a 3×3 convolution, batch normalisation, ReLU and 2×2 max pooling, followed by global average pooling and a small dense head (Figure 3). The number of filters doubles from block to block (32, 64, 128) while the feature maps shrink from 224 to 28 pixels per side. In total the model has 111,145 parameters (Table 5).

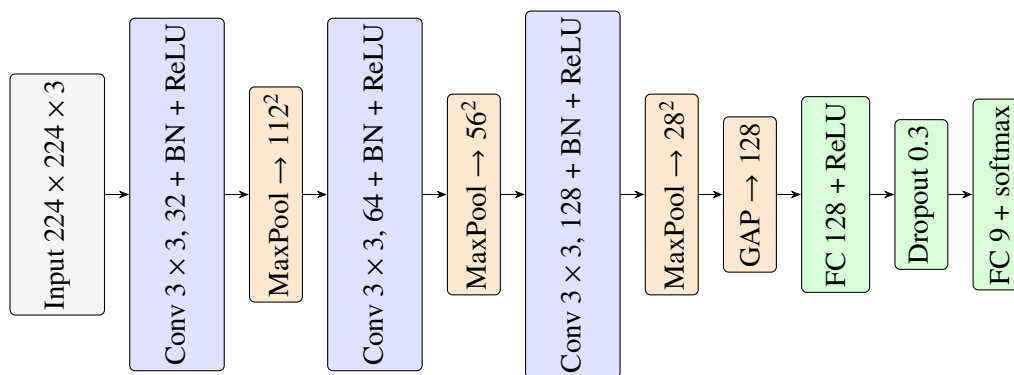


Figure 3: Architecture of E1 SimpleCNN. Blue: convolution, orange: pooling, green: fully connected part.

Table 5: E1 layers, output shapes and parameter counts.

Layer	Output shape	Parameters
Input	$224 \times 224 \times 3$	0
Conv 3×3 (32), BN, ReLU, MaxPool	$112 \times 112 \times 32$	928
Conv 3×3 (64), BN, ReLU, MaxPool	$56 \times 56 \times 64$	18,560
Conv 3×3 (128), BN, ReLU, MaxPool	$28 \times 28 \times 128$	73,984
Global average pooling	128	0
Dense 128, ReLU, Dropout 0.3	128	16,512
Dense 9 (logits)	9	1,161
Total		111,145

The convolutions have no bias, since the batch normalisation right after them already adds one. A useful way to see the limits of this design is the receptive field: after three blocks, one value of the last feature map depends on a patch of only 22×22 input pixels. That is enough to recognise a texture but not the outline of a bottle or a box, so E1 should have trouble with classes that are recognised by their shape.

3. Complex CNN: MultiScaleCNN (E2)

For the complex model we started from a question raised in the lecture on modern CNNs: what is the right size for a convolution kernel? The lecture's answer, taken from GoogLeNet, is that combining several sizes can work better than picking one [2, 6]. This fits our data. A crumpled plastic bag is recognised by fine wrinkles and reflections, which a small kernel picks up, while a cardboard box is recognised by large flat faces and straight edges. So each block of our network runs a 1×1 , a 3×3 and a 5×5 convolution side by side on the same input and stacks their outputs (Figure 4). Half of the output channels go to the 3×3 branch and a quarter to each of the others. Compared with the Inception block in the lecture we left out the extra 1×1 reductions and the pooling branch to keep the model simple.

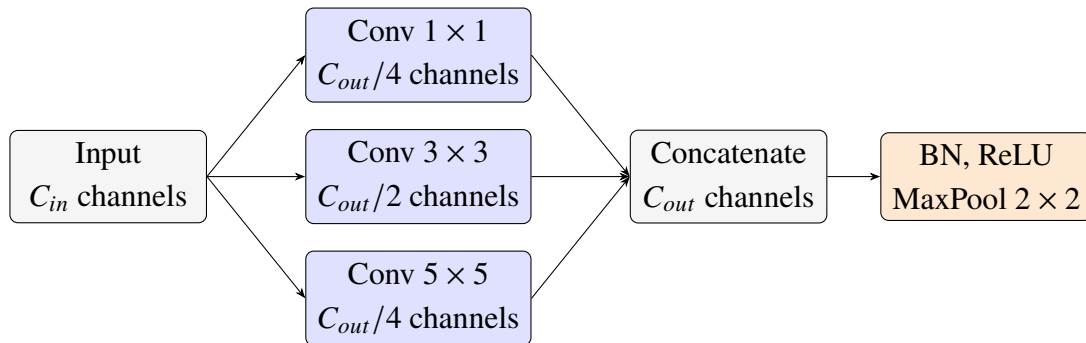


Figure 4: One multi-scale block. The three branches see the same input and are joined along the channel axis. Blocks 1 to 4 use $C_{out} = 64, 128, 256$ and 256 .

The full network (Figure 5) starts with a single 3×3 convolution as a stem, then has four multi-scale blocks, each followed by max pooling, and ends with the same kind of head as E1. It has 1,230,377 parameters, about eleven times more than E1 (Table 6).

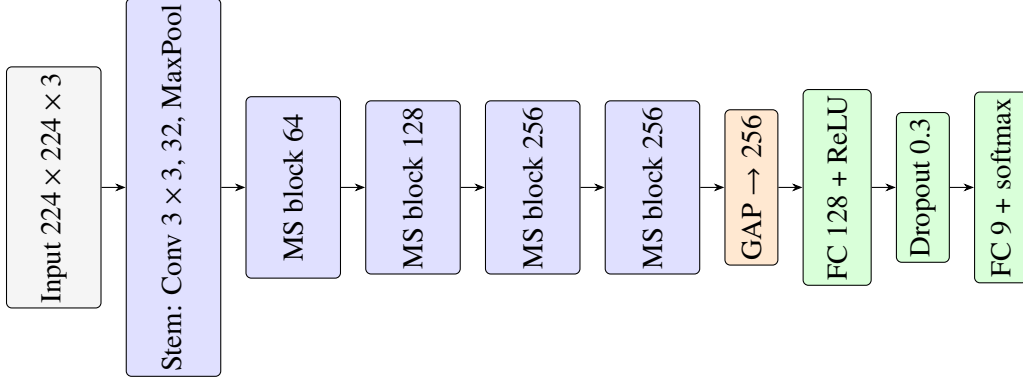


Figure 5: Architecture of E2 MultiScaleCNN. Every multi-scale (MS) block is followed by 2×2 max pooling, so the feature maps go from 112 to 7 pixels per side.

Table 6: E2 stages. Channel counts in brackets are the $1 \times 1 / 3 \times 3 / 5 \times 5$ branches.

Stage	Output shape	Parameters
Stem: Conv 3×3 , 32, BN, ReLU, MaxPool	$112 \times 112 \times 32$	928
MS block 1 (16 / 32 / 16), MaxPool	$56 \times 56 \times 64$	22,656
MS block 2 (32 / 64 / 32), MaxPool	$28 \times 28 \times 128$	90,368
MS block 3 (64 / 128 / 64), MaxPool	$14 \times 14 \times 256$	360,960
MS block 4 (64 / 128 / 64), MaxPool	$7 \times 7 \times 256$	721,408
Global average pooling	256	0
Dense 128, ReLU, Dropout 0.3	128	32,896
Dense 9 (logits)	9	1,161
Total		1,230,377

Through its 5×5 branches the last block of E2 sees a 154×154 pixel area of the input, against 22×22 for E1, so it can use the shape of the whole item. One caveat: E2 is not only multi-scale but also deeper and wider than E1. If E2 wins, we can only credit the design as a whole, not the multi-scale branches by themselves.

4. Transfer Learning and Fine-Tuning: EfficientNet-B0 (E3)

For transfer learning we chose EfficientNet-B0 [5], mainly because it is accurate for its size and trains quickly on the free Colab GPU we had. With its original 1,000-class head it has 5.3 million parameters and reaches 77.7 % top-1 accuracy on ImageNet-1K [11]. The network is built from MBConv blocks. Each block widens the channels

with a 1×1 convolution, applies a cheap depthwise 3×3 or 5×5 convolution (one filter per channel), reweights the channels with a squeeze-and-excitation step and narrows them again with another 1×1 convolution. We removed the ImageNet head and put Dropout(0.2) and a $1280 \rightarrow 9$ dense layer in its place, which gives 4,019,077 parameters (Figure 6).

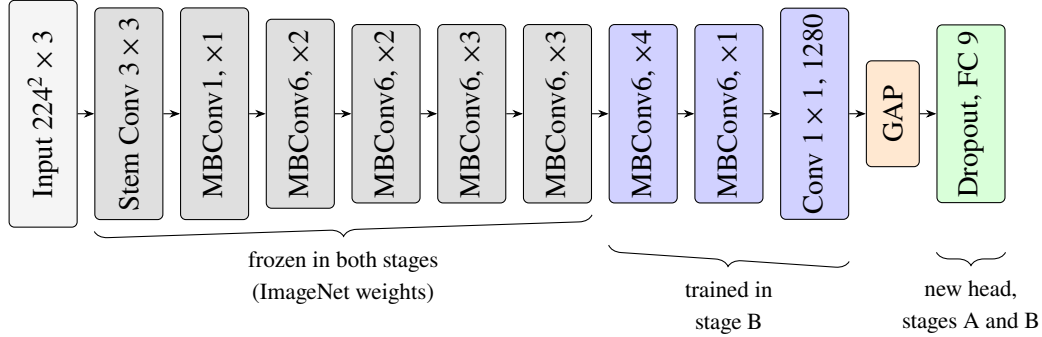


Figure 6: E3: EfficientNet-B0 with a new 9-class head. Grey blocks keep their ImageNet weights; the three blue blocks (features [6:], all at 7×7 resolution) are fine-tuned in stage B.

Training happens in two stages that match the transfer and fine-tuning phases of the lecture [3] (Table 7). In stage A the whole backbone is frozen, including the running statistics of its batch normalisation layers, and only the new head learns. This is feature extraction: the backbone acts as a fixed feature generator and the head learns to read its 1,280 outputs. Stage B starts from the best stage-A checkpoint and also unfreezes the last three blocks, which hold the most task-specific features. We kept the first six blocks frozen because they detect generic edges and textures that should work for waste photos too, and because this keeps each epoch short. Whichever stage reaches the higher validation macro-F1 becomes the final E3 model.

Table 7: The two training stages of E3.

	Stage A: feature extraction	Stage B: fine-tuning
Starts from	ImageNet weights, new random head	best stage-A checkpoint
Trainable part	head only	last 3 backbone blocks and head
Trainable parameters	11,529	3,167,269 (79 %)
Learning rate (AdamW)	10^{-3}	10^{-4}
Maximum epochs	10	15

E3 is different from E1 and E2 in its pretrained weights, its architecture and its training schedule. Comparing them therefore tells us how transfer learning does against training from scratch, not which architecture is better.

5. Training Protocol

Table 8 lists the training settings. We fixed them before training any model and used the same values for all models trained from scratch; E3 only differs where its two stages require it.

Table 8: Training settings.

Setting	E1, E2, E4	E3
Loss	cross-entropy, no class weights	same
Optimiser	AdamW, lr 10^{-3} , weight decay 10^{-4}	AdamW, lr 10^{-3} (stage A) and 10^{-4} (stage B), weight decay 10^{-4}
Learning-rate schedule	halved after 2 epochs without a better validation macro-F1	same, per stage
Batch size	32	32
Maximum epochs	40	10 + 15
Early stopping	5 epochs without a better validation macro-F1	same, per stage
Checkpoint kept	best validation macro-F1	best of both stages
Precision	mixed (16-bit) on GPU	same

A few of these choices came out of the project itself. We first gave E1 a limit of 30 epochs, but its validation score was still rising at epoch 30, so we raised the limit to 40 for both models trained from scratch and let the same run continue. We also considered weighting the loss by class frequency. We decided against it after E2 had been trained, because the deeper model already recovered most of the small classes on its own (validation recall for Textile Trash went from 29 % with E1 to 51 % with E2, and for Miscellaneous Trash from 30 % to 70 %).

We select models by validation macro-F1 rather than accuracy, both for early stopping and for picking the epoch to keep. With unequal classes, macro-F1 gives Textile Trash the same weight as Plastic.

We did not run a hyperparameter search. We had planned a small one (two learning rates and then two values of one more setting for each model), but it did not fit into the deadline. The values in Table 8 are common defaults for AdamW and for fine-tuning. They may not be the best for every model, E1 in particular.

What we did instead was to repeat training. A single run depends on chance: the starting weights, the order of the batches, which augmentations are drawn and which neurons dropout switches off. So E1, E2 and E3 were each trained three times, with

seeds 42, 43 and 44, and we report the mean and standard deviation (SD). The split stays the same in all runs, so the SD shows how much the result moves with training randomness, not with the choice of test images. E4 was trained once.

All training ran on Google Colab with an NVIDIA T4 GPU, about 40 seconds per epoch for E1 and E3 and 45 seconds for E2. In practice the slowest part was reading the images from Google Drive: the first epoch of a new session took around 15 minutes, until we started copying the dataset to the Colab disk at the beginning of each session.

IV. Experimental Results

1. Metrics

We report accuracy and the macro averages of precision, recall and F1. For a class k , precision P_k is the share of images predicted as k that really belong to k , and recall R_k is the share of the real k images that the model found. F1 combines the two, and the macro average gives every class the same weight:

$$F1_k = \frac{2 P_k R_k}{P_k + R_k}, \quad \text{Macro-F1} = \frac{1}{K} \sum_{k=1}^K F1_k. \quad (4)$$

Macro precision and macro recall are averaged in the same way. Accuracy, the share of all images classified correctly, is easier to read but can hide a weak small class, which is why we use macro-F1 to compare models.

2. Results on the Test Set

[TODO: fill Table 9 from 04_Results/final/model_comparison.csv after notebook 09 has run with all three seeds, then delete this note.]

Table 9: Test set (707 images). E1 to E3: mean $\pm$ SD over seeds 42, 43 and 44. E4: one run. Precision, recall and F1 are macro averages.

Model	Accuracy	Precision	Recall	F1-score
E1 SimpleCNN	TBD	TBD	TBD	TBD
E2 MultiScaleCNN	TBD	TBD	TBD	TBD
E3 EfficientNet-B0 (fine-tuned)	TBD	TBD	TBD	TBD
E4 MultiScaleCNN, no augmentation	TBD	TBD	TBD	TBD

[TODO: per-class test F1 from 04_Results/final/per_class_f1.csv, one sentence on the hardest classes.]

Missing figures/E1_confusion_matrix.png Drive: 04_Results/experiments/E1/	Missing figures/E2_confusion_matrix.png Drive: 04_Results/experiments/E2/
Missing figures/E3_confusion_matrix.png Drive: 04_Results/experiments/E3/	Missing figures/E4_confusion_matrix.png Drive: 04_Results/experiments/E4/

Figure 7: Test confusion matrices of the seed-42 runs (rows: true class, columns: predicted class).

3. Validation Results and Training Behaviour

Table 10 shows the validation scores of the seed-42 runs, the scores we used to pick each checkpoint. On the 719 validation images the ranking was clear: E1 reached a macro-F1 of 0.650, E2 0.787 and E3 0.879, so each step up added roughly 0.1.

Table 10: Validation set (719 images), seed-42 runs, selected checkpoints.

Model	Best epoch	Accuracy	Precision	Recall	F1-score
E1 SimpleCNN	33 of 38	0.656	0.706	0.636	0.650
E2 MultiScaleCNN	35 of 40	0.787	0.815	0.772	0.787
E3 EfficientNet-B0	25 of 25	0.880	0.894	0.871	0.879
E4 no augmentation	29 of 34	0.814	0.819	0.803	0.808

[TODO: add the validation macro-F1 mean $\pm$ SD of the three seeds from each selection.json.]

The training curves (Figure 8) explain part of this ranking. E1 kept improving slowly until about epoch 33, and its training and validation curves stayed close the whole time. It was not overfitting, it simply did not have enough capacity. E2 behaved

differently: after roughly epoch 30 its training loss kept going down (0.49 at the end) while the validation loss stayed around 0.64, a mild overfit that is harmless because we keep the best validation epoch anyway. E3 was the most interesting case. With only the head trained, stage A already reached 0.810, better than our best model trained from scratch. Fine-tuning in stage B raised this to 0.879, but by the end of stage B the training accuracy was close to 100 % (training loss 0.02 against 0.42 on validation), so we saw no point in training it longer.

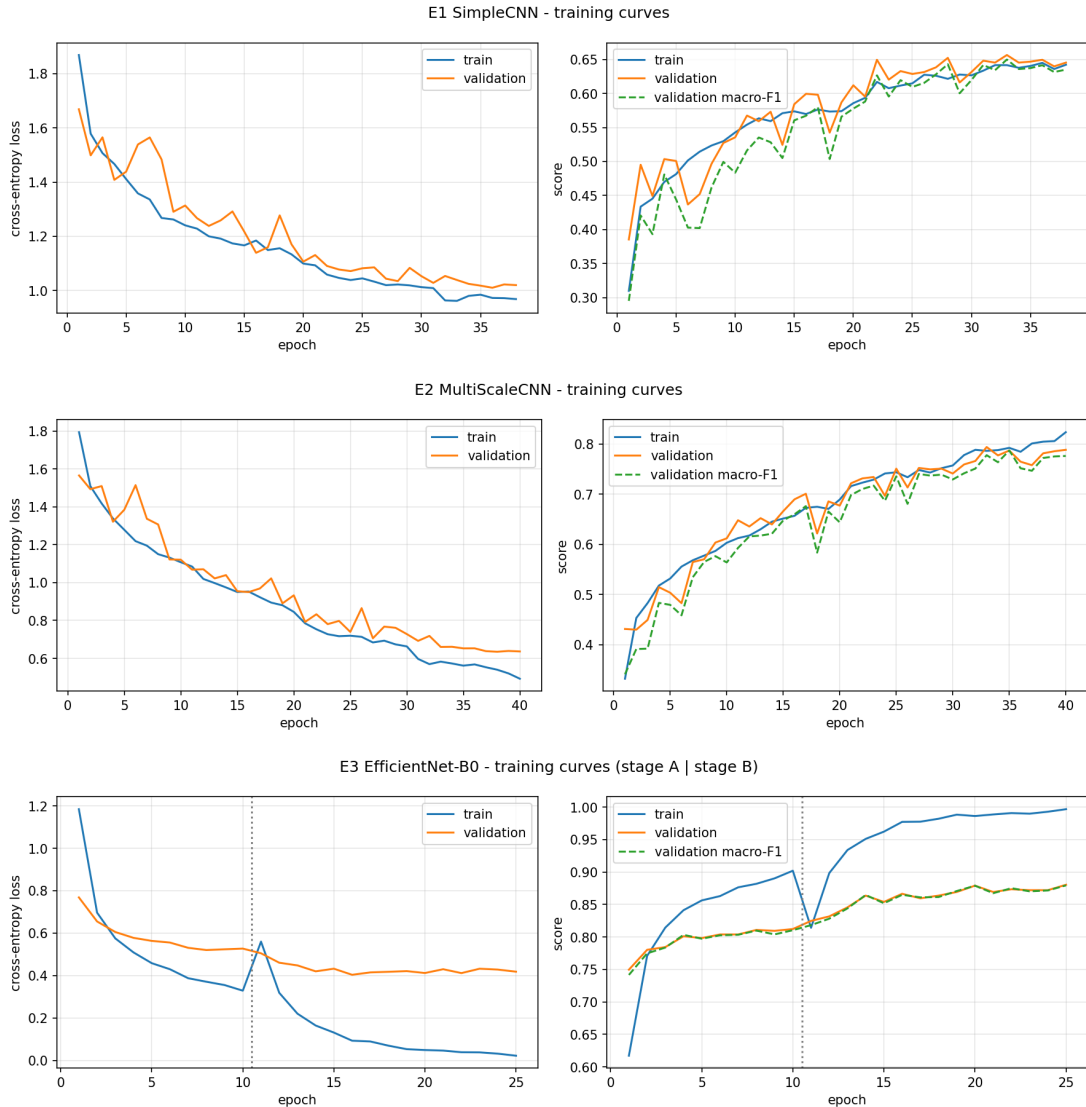


Figure 8: Training and validation loss, accuracy and macro-F1 per epoch for E1, E2 and E3 (seed 42).

4. Augmentation Ablation (RQ4)

The ablation result goes against the usual expectation. Without augmentation, the multi-scale CNN (E4) reached 0.808 validation macro-F1, slightly more than the 0.787 of the same model with augmentation (E2). We do not think this shows that augmentation hurts. The gap is 0.021 from a single run of each model, and the validation macro-F1 of one model often moved by up to 0.04 from one epoch to the next. The loss curves tell a clearer story: without augmentation the training loss fell to about 0.22 while the validation loss stayed near 0.57, whereas with augmentation the two stayed much closer (0.49 and 0.64). Augmentation clearly regularised the model; it just did not turn this into a better score within 40 epochs.

We can think of three reasons, none of which we tested. The photos are already very uniform (same camera angle, same floor, centred objects), so random crops and rotations add variety that the test images do not have. Colour jitter may blur colour differences that matter for classes such as Cardboard or Vegetation. And augmented training usually needs more epochs than we gave it. **[TODO: compare E2 and E4 on the test set, keeping in mind that E4 has one run.]**

5. Error Analysis

Looking at the validation errors, one pattern stood out for every model: when unsure, the models tended to answer “Plastic”. For E1 this was extreme. It predicted Plastic for 249 validation images, of which only 116 really were Plastic, so 54 % of all its errors were wrong Plastic predictions. The share dropped to 33 % for E2 and 31 % for E3. Plastic is the largest class and includes almost every shape, colour and degree of shine, which probably makes it the safest guess for a model that has not learned the other classes well. Table 11 lists the most common confusions.

Table 11: Most frequent confusions on validation (true → predicted, number of images).

Model	Confusions
E1	Metal → Plastic 34, Miscellaneous → Plastic 31, Glass → Plastic 25, Cardboard → Plastic 18
E2	Plastic → Metal 23, Cardboard → Plastic 17, Glass → Plastic 14, Textile → Miscellaneous 13
E3	Miscellaneous → Plastic 9, Glass → Plastic 8, Metal → Plastic 6, Cardboard → Metal 5

The two hardest classes were Miscellaneous Trash and Textile Trash. E1 found only 30 % and 29 % of them. E2 found 70 % and 51 %, and E3 71 % and 82 %. Both classes are collections of very different objects rather than one material, and Textile Trash is also the smallest class, so these numbers are not surprising.

[TODO: test error analysis: take the model with the best mean validation macro-F1, open its `errors.csv`, describe the failure modes of 50 to 100 misclassified test images grouped by confusion pair, and add a figure with example errors.]

V. Conclusion

[TODO: the findings below come from validation. Replace each number with the test mean $\pm$ SD and keep a finding only if the test results still support it.]

The fine-tuned EfficientNet-B0 (E3) was clearly our best model, with a validation macro-F1 of 0.879, compared with 0.787 for our multi-scale CNN (E2) and 0.650 for the simple CNN (E1). Going back to the research questions:

RQ1. A three-block CNN trained from scratch reaches a macro-F1 of 0.650. It does not overfit, yet it misses most Textile Trash and Miscellaneous Trash items, which suggests that it is too small for the task rather than short of data.

RQ2. The multi-scale CNN improves macro-F1 by 0.137 and recovers much of the two hard classes. As E2 is also deeper and wider than E1, we credit this gain to the design as a whole.

RQ3. ImageNet pretraining adds about 0.09 macro-F1 on top of E2. The most telling result is that the frozen pretrained features with only a new head (0.810) already beat both models trained from scratch.

RQ4. Removing augmentation changed validation macro-F1 by +0.021, which is within the noise of a single run. Augmentation did narrow the gap between training and validation loss, but our experiment cannot show that it changes accuracy.

The errors that remain are mostly confusions between materials that look alike (glass, metal and plastic; cardboard and plastic), Plastic used as a fallback answer, and the two mixed classes.

The study has clear limitations. Photos of one object can still end up in different subsets when they straddle a block boundary or are far apart in numbering, so the test scores may be a little optimistic. We did not search for better hyperparameters, and E1 in particular might do better with other settings. The SD over three seeds

only covers training randomness: all runs share one split, and E4 was run only once. Our comparisons also change more than one thing at a time, since E2 differs from E1 in several ways and E3 differs from both in pretraining and schedule. Last, all images come from one setup with the same floor and camera angle, and we expect lower accuracy on photos taken elsewhere, for example on a real sorting line.

For a practical system with a small labelled dataset, our results point towards starting from a pretrained network rather than designing a CNN from scratch: in our tests it was the most accurate model and needed only 25 epochs. Before using it in the field, though, it would have to be trained and tested on images from the place where it will run.

References

- [1] N. T. K. Ngan, “Convolutional Neural Network (CNN),” lecture slides, Chapter 5.1, Deep Learning course, National Economics University, 2026.
- [2] N. T. K. Ngan, “Modern Convolutional Neural Networks,” lecture slides, Chapter 5.2, Deep Learning course, National Economics University, 2026.
- [3] N. T. K. Ngan, “CNN Techniques,” lecture slides, Chapter 5.3, Deep Learning course, National Economics University, 2026.
- [4] S. Single, S. Iranmanesh, and R. Raad, “RealWaste: A novel real-life data set for landfill waste classification using deep learning,” *Information*, vol. 14, no. 12, art. 633, 2023. <https://www.mdpi.com/2078-2489/14/12/633>. Dataset: UCI Machine Learning Repository, <https://archive.ics.uci.edu/dataset/908/realwaste>.
- [5] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. ICML*, 2019. arXiv:1905.11946.
- [6] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, “Going deeper with convolutions,” arXiv:1409.4842, 2014.
- [7] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” arXiv:1502.03167, 2015.
- [8] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

- [9] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019. arXiv:1711.05101.
- [10] O. Russakovsky, J. Deng, H. Su, et al., “ImageNet large scale visual recognition challenge,” arXiv:1409.0575, 2014.
- [11] PyTorch, “torchvision efficientnet_b0, weights IMAGENET1K_V1,” model documentation. https://docs.pytorch.org/vision/main/models/generated/torchvision.models.efficientnet_b0.html.